Final Engineering Report: secanalyzer

Milestone 5 Final Written Report

CSC 366 Security-Focused Course Project

Repository: https://github.com/flufcloud/366_project

Project board: https://github.com/users/flufcloud/projects/2/views/1

Prepared by Siddhu Bhimireddy

This report summarizes the full project lifecycle for secanalyzer, from the initial requirements and threat model through the final release. secanalyzer is a local command-line tool that helps a developer understand a codebase and review security-related GitHub work. The project began with a broader AI-agent idea for reproducing mobile application bugs. During planning, the work was narrowed to a safer and more realistic local security tool. That change is part of the lifecycle described below.

1. Executive Summary

secanalyzer is a local software tool for developers and security engineers. The user runs it from a terminal on Linux or WSL. The tool can scan a local repository, produce a Markdown report, connect to GitHub to list open issues and pull requests, and ask an AI model to help classify selected items as low, medium, or high security risk.

The most important design decision is that secanalyzer is not a cloud service. There is no secanalyzer server, no database, and no telemetry system. The tool runs on the user's machine. It only makes outside network calls when the user chooses features that need them: GitHub for issue and pull request data, and an optional AI provider such as Anthropic or Google for analysis.

The system is useful because many developers have to review unfamiliar repositories or long lists of GitHub issues. A tool like this can give them a structured starting point. It does not replace a human reviewer. It organizes information, points to likely risks, and suggests next steps that a human can check.

Security shaped the project from the beginning. The tool handles local source code, GitHub access tokens, and AI API keys. Those are sensitive. The final release includes several controls: local credential storage, hidden token entry, repository path checks, file extension allowlists, secret redaction, prompt-size limits, structured AI responses, local operational logs, and automated checks in CI.

The final evaluation passed the selected release gates. The automated test suite reported 99 passed tests and 1 skipped platform-specific test. Bandit did not report issues in the application source. pip-audit did not report known dependency vulnerabilities. The CLI help command and a static scan smoke test both completed successfully.

The release still has limits. Namely, AI output can be wrong, chances of prompt injection is reduced but not fully solved, and secret detection is based on known patterns and can miss new formats. Additionally, long AI runs can be slowed by provider rate limits. These gaps are documented in the issue log and roadmap.

Question	Final answer
What was built?	A local CLI that scans repositories and helps triage GitHub issues and pull requests for security review.
Who is it for?	Developers, security reviewers, and technical writers working with source repositories.
What leaves the machine?	Only user-authorized GitHub requests and optional AI requests containing bounded, redacted content.
What proves release readiness?	Tests, static analysis, dependency audit, smoke tests, documentation, and issue tracking for known gaps.

2. Initial Project Design and Scope Change

The initial project design was titled "Automating Bug Reproduction in Software Development with AI Agents." The original idea focused on a mobile application workflow. A user would choose a GitHub issue, then an AI agent would launch the application inside a virtual machine, follow the issue's reproduction steps, interact with the graphical interface, and record a final screenshot. A vision-language model would then judge whether the bug was reproduced.

That idea was valuable because bug reproduction is a real engineering problem. Before a team can fix a bug, it needs confidence that the bug is caused by the software and not by a user mistake, network problem, or local machine issue. Automating part of that process could save time, especially for user-interface bugs that require many manual clicks.

The original design also raised serious concerns. An AI agent that can use a mouse, keyboard, terminal, and virtual machine has a large amount of power. It could delete files, leak private data, or damage the test environment if it follows bad instructions. It would also require a stable mobile test environment, a reliable graphical agent, and a judge model that can accurately evaluate screenshots. That scope was too large for one course project and created security risks that would be difficult to contain well.

The project was narrowed to a safer form of automation. Instead of letting an agent control a graphical application, the final system helps a human reviewer inspect code and GitHub work. The tool still uses AI, but the AI is not allowed to operate the computer or change the repository. It reads bounded text and returns advisory analysis. The human remains in control of what to run and what to trust.

This change kept the core motivation of the first idea: reduce repetitive review work and help engineers focus on likely problems. It also made the security model clearer. The final system has fewer actions it can take, fewer places where untrusted instructions can cause harm, and a simpler path to testing and documentation.

3. Final System Overview

The final release supports two main workflows. The first workflow is the LLM codebase analyzer. In this workflow, the user runs secanalyzer against a local repository and asks the tool to generate a security-focused report. The tool reads the repository in smaller parts rather than sending the entire project at once. It skips files and folders that are not useful for review, checks for common secret patterns, limits how much content is sent in each request, and produces a structured report. The output includes per-file notes, intermediate summaries, and a final written report. This design makes the report easier to review because the user can see how the final analysis was built from smaller pieces of evidence.

The second workflow is GitHub issue and pull request triage. In this workflow, the user stores a GitHub token and an AI provider key, then asks secanalyzer to fetch open issues and pull requests from a GitHub repository. The result is a structured risk assessment that explains the likely severity of the issue and suggests possible next steps. This workflow can also be augmented with the security report from the first workflow, so the issue or pull request analysis can be informed by the broader structure and security concerns of the codebase.

The tool is packaged as a Python 3.11+ project. It uses pyproject.toml and uv.lock for reproducible setup. It can be run as the console command secanalyzer or with python -m secanalyzer. The expected deployment is a developer's local environment, not a hosted service.

Workflow	What the user does	What the tool returns
Local scan	Run secanalyzer --scan PATH.	A Markdown inventory and security-oriented repository summary.
GitHub triage	Run secanalyzer --issues owner/repo and select an item.	A risk level, explanation, relevant locations when available, and suggested mitigation.
Deep report	Run secanalyzer --llm-report PATH with an output file or report directory.	A fuller AI-assisted security report with supporting artifacts.

4. Development Lifecycle

The work was organized around milestones. The early requirements milestone defined the main users, use cases, non-functional requirements, and risks. The design milestone turned those requirements into system components and a threat model. The beta milestone delivered a working command-line tool with repository scanning, GitHub integration, AI analysis, tests, and CI. The final milestone focused on hardening, documentation, evaluation evidence, and accepted risks.

The implementation followed a small-module design. The command-line module handles user commands. The configuration module reads and writes tokens. The repository analyzer handles local file scanning. The GitHub client fetches issue and pull request data. The AI layer builds prompts, calls the selected provider, and checks the response. The session and output modules connect those parts for the user. This structure made the project easier to test because each part has a clear responsibility.

Testing was added as the system grew. Network calls were mocked in tests so the suite could run without depending on GitHub or an AI provider. The tests cover command parsing, credential paths, repository scanning, redaction, GitHub response handling, AI retry behavior, report generation, and package entry points.

The final release added production-readiness work. Documentation was organized into guides and reports. The operations module was added to write sanitized JSONL events for command lifecycle, scan counts, redaction hits, API failures, and retry pressure. The issue log was updated with known gaps, including model rate limiting, stronger secret detection, prompt-injection evaluation, and future supply-chain provenance work.

Milestone	Main result
Requirements	Defined the users, MVP features, security concerns, and backlog.
Design and threat model	Mapped the system into components and identified main attack surfaces.
Beta release	Delivered a working CLI with scanning, GitHub triage, AI analysis, tests, and CI.
Final release	Added hardening, operational logging, final validation evidence, documentation, and issue tracking.

5. Architecture

The system can be understood as six cooperating parts. The first part is the command-line interface. It reads the user's command, checks whether the command makes sense, and routes the request. The second part is configuration. It stores tokens and API keys in the user's local config area and avoids printing them back to the terminal.

The third part is the repository scanner. It reads the local project folder under strict rules. It does not follow symlinks during the scan. It checks that each file remains under the selected root folder. It only reads allowed text file types. It skips common noisy folders such as dependency directories and builds outputs. It redacts common secret patterns before content is shown or sent onward.

The fourth part is the GitHub client. It asks GitHub for open issues and pull requests. For pull requests, it can also fetch changed file information and patch text. The patch text is capped so that one large pull request cannot consume too much memory or too much AI context.

The fifth part is the AI orchestration layer. This part prepares the prompt, marks outside content as untrusted data, checks that the prompt is not too large, scans for secret-like text before sending, calls the selected AI provider, and checks that the response follows the expected format. A fixed response format matters because the tool should not treat random model text as a trusted result.

The sixth part is output and session handling. It writes reports to files or stdout, shows the interactive GitHub menu, and prints the final risk summary in a readable form. The output is designed for a human reviewer, not for automatic code changes.

Part	Plain-language role
CLI	Receives the user command and starts the right workflow.
Config	Stores tokens locally and checks credential status.
Repository scanner	Reads local files safely and produces a report-ready inventory.
GitHub client	Fetches issue and pull request data from GitHub.
AI layer	Prepares safe prompts, calls the selected model, and validates the response.
Output/session	Shows menus and writes the final reports.

6. Security Analysis & Threat Modeling

The threat model focused on where untrusted data enters the system, where sensitive data could leave it, and which generated outputs need protection after they are created. The main entry points are command-line arguments, local repository files, GitHub issue text, GitHub pull request patches, AI responses, and credential files. The main outside services are GitHub and the selected AI provider.

The first major risk is prompt injection. A malicious issue or pull request can contain text such as “ignore the previous instructions.” The tool reduces this risk by placing GitHub-controlled content inside clear data boundaries and by asking the model for a fixed JSON-style answer. This helps separate instructions from evidence. It does not fully solve the problem, because AI models can still be influenced by malicious text.

The second major risk is credential leakage. GitHub tokens and AI keys should not appear in reports, logs, prompts, or errors. The tool uses hidden input when storing keys, does not intentionally print them, sends them only as part of API authentication, and uses user-facing errors rather than raw stack traces. For issue analysis, the full prompt is checked before sending; if a secret-like pattern remains, the request is stopped.

The third major risk is unsafe local file handling. A repository might include unusual paths or filenames. The scanner resolves the selected root folder, checks that files stay inside that folder, avoids following symlinks during the scan, and does not run shell commands using filenames. This reduces the chance that a malicious repository can make the tool read unrelated local files or execute commands.

The fourth major risk is accidental transfer of sensitive code to an AI vendor. The tool cannot know every type of private business logic, so it focuses on practical limits: allowed file types, skipped directories, per-file caps, prompt-size caps, redaction, and clear documentation. Users can also choose to use only the local scan mode, which does not require sending code to an AI provider.

The fifth major risk is dependency compromise. The project uses a pinned uv.lock file, installs dependencies in CI with frozen synchronization, runs pip-audit for known vulnerabilities, and runs Bandit for risky Python patterns. These checks reduce risk, but they do not replace future work such as SBOM generation or stricter review rules for dependency changes.

The sixth major risk is exposure of the generated security report itself. The final report is not just a normal output file. It may contain a summary of the codebase structure, trust boundaries, risky files, likely weaknesses, and recommended fixes. This makes it useful to the development team, but it also makes it useful to an attacker. Even if the report does not include raw secrets, it can still reveal where the system is most likely to fail. For this reason, the report should be treated as a sensitive engineering artifact. It should be stored in a controlled location, shared only with people who need it, and excluded from public commits unless it has been reviewed and approved.

Risk	Current control	What remains
Prompt injection	Untrusted data boundaries and fixed response validation.	The model can still be biased by malicious content.
Credential leakage	Hidden input, local config files, no intentional echo, prompt pre-send checks, short errors.	A compromised machine or future logging bug could still expose secrets.
Unsafe file handling	Path confinement, no symlink following, no shell-based scan.	Unusual filesystem edge cases remain possible.
Sensitive code transfer	Local-only scan option, redaction, caps, allowlists, and clear documentation.	Private logic may still be sent if the user chooses AI features.
Supply-chain risk	Locked dependencies, frozen CI install, Bandit, and pip-audit.	No SBOM or package provenance process yet.
Exposure of the generated security report	Report contents are written locally and can be reviewed before sharing. Documentation should warn users to treat reports as sensitive.	The report may still be copied, committed, or shared too broadly by mistake.

7. Evaluation and Results

The final evaluation used automated tests, static analysis, dependency auditing, and smoke tests. The goal was to check that features worked and to check that the final release behaved like a tool another person could build and run. I also evaluated the results of the security artifact itself and cross-referenced with existing issues documented externally in a case study of the Open Wearables repository, by The Momentum.

The pytest suite reported 99 passed tests and 1 skipped test. The tests cover command behavior, configuration, repository scanning, redaction, GitHub integration through mocked responses, AI helper logic, retry behavior, report generation, and package startup. Mocking network calls was important because CI should not depend on live GitHub or AI service availability.

Bandit was run against the application source. It reported no issues. pip-audit was run against the dependency set. It reported no known vulnerabilities, with the local project skipped because it is not a published package on PyPI. These tools do not prove the system is secure, but they are useful release gates because they catch common problems before delivery.

Two smoke tests checked the installed user path. The help command printed successfully, which confirms the CLI entry point can run. A static scan of the repository wrote a Markdown output file, which confirms that the main local workflow works outside of unit tests.

The release also evaluated operational behavior. Logs record command lifecycle events, scan counts, redaction counts, API errors, and retry pressure. The logs are useful for debugging without storing raw prompts or secrets. The tool also reports warnings for redaction and truncation, which helps a user understand when the report was limited for safety.

I also compared the LLM-assisted reports and issue triaging against documented failure points in the Open Wearables repository. This comparison helped test whether secanalyzer could identify the same broad security themes that a human-written review found. The result was mixed. The LLM report broadly identified the major categories of security risk, including the general areas of the system that deserved attention. However, it was less reliable at the implementation level. Some specific technical details were incomplete, too general, or not directly tied to exact code behavior. This suggests that the tool is useful for early review and prioritization, but not as a replacement for human inspection. A likely improvement would be using a stronger model for deeper code reasoning, especially when the goal is to identify precise implementation-level flaws.

Check	Result	Meaning
pytest	99 passed, 1 skipped	Main code paths and mocked network paths passed automated tests.
Bandit	No issues identified	Static analysis did not flag dangerous Python patterns in source.
pip-audit	No known vulnerabilities found	Known-vulnerability scan passed for locked dependencies.
CLI help	Successful	The installed command can run and show usage.
Static scan smoke test	Successful	The tool can scan a repository and write a report.
Security artifact usefulness	Mixed result	The LLM report offered less reliable implementation-level analysis. A stronger frontier model could improve precision.

8. Deployment, Operations, and CI/CD

Deployment is local. A third party can clone the repository, install dependencies with uv sync --all-groups, and run uv run secanalyzer --help. The project can also produce a wheel and source distribution with uv build. Because there is no server, deployment does not involve cloud hosting or database setup.

The CI pipeline uses GitHub Actions. On pushes and pull requests to main or master, the workflow checks out the code, installs uv, installs the locked dependency set with uv sync --frozen --all-groups, runs pytest, runs Bandit, and runs pip-audit. This gives the project a repeatable gate for test and security checks.

The documentation supports deployment readiness. The README explains prerequisites, setup, commands, and troubleshooting. The quickstart guide gives a short first-run path. The deployment guide explains how to build, configure, monitor, and run the application. The security guide explains how data is handled and how vulnerabilities should be reported. The technical and security reports connect the design to the implementation.

Operations are intentionally simple. The tool writes local sanitized logs. A user can control logging through environment variables. The logs are useful for troubleshooting local runs, but they are not a centralized monitoring platform. That choice matches the project's local CLI design.

The documentation also notes environment-specific issues. For example, a virtual environment created in Windows should not be reused inside WSL. The recommended fix is to keep separate environments or remove the existing .venv and run uv sync again in the environment being used.

- Build and install: uv sync --all-groups
- Run tests: uv run pytest
- Run static analysis: uv run bandit -r src/secanalyzer
- Run dependency audit: uv run pip-audit
- Confirm CLI availability: uv run secanalyzer --help
- Run local scan: secanalyzer --scan PATH -o report.md

9. Concluding Engineering Analysis

The final system is a local security-review assistant for developers who need help understanding a repository and prioritizing GitHub work. The project began with a different idea: an AI agent that could reproduce bugs in a graphical application. That first design was interesting, but it created a much larger security problem because the agent would need to control a computer, open applications, and interact with a user interface. During development, the project moved toward a safer and more practical final scope. Instead of giving an AI agent broad control over a machine, the final tool keeps the user in control and limits the AI's role to review, summarization, and risk assessment.

This scope change improved the engineering shape of the project. The final system has clearer boundaries. The local machine, local repository, credential files, GitHub, and AI provider are separated in the design. Each boundary has a related control: local file scanning uses path confinement and skip rules, credentials are stored locally and not intentionally printed, GitHub content is treated as untrusted input, and AI prompts are bound and checked before they are sent. These choices do not remove all risk, but they make the system easier to reason about and safer to operate.

The strongest part of the final release is its structure. The codebase is divided into separate modules for command-line handling, configuration, repository scanning, GitHub access, AI communication, report generation, and output. This makes the system easier to test and easier to explain. It also supports the security design because sensitive responsibilities are not mixed together in one large file. For example, credential handling is kept separate from report writing, and GitHub fetching is kept separate from AI prompt construction.

The evaluation results show that the final release is stable enough for a course production release. The automated test suite passed with 99 tests passing and 1 skipped. Bandit did not identify dangerous Python patterns in the source code. pip-audit did not find known vulnerabilities in the locked dependencies. The CLI help command ran successfully, and the static scan smoke test confirmed that the tool can scan a repository and write a report. These results do not prove that the tool is secure in every case, but they do show that the main workflows work and that basic security checks are passed.

The project also evaluated the generated security report as an artifact. This was important because the report is one of the main products of the tool. The tool was validated on the Open Wearables repository by The Momentum. The results showed that the LLM-assisted report could identify broad security themes, but its implementation-level details were mixed. This means the tool is useful for early review, orientation, and prioritization, but it should not be treated as a replacement for human code review. A stronger model could improve the accuracy of detailed findings, especially when the system needs to connect a risk theme to exact implementation behavior. In particular, a strong reasoning model such as Opus 4.8 could really improve outputs.

The main engineering trade-off was between usefulness and safety. The simplest way to build this system would be to send a large amount of repository content to an AI model and ask for a full report. That approach would be easier to implement, but it would be more expensive, less reliable, and more likely to expose sensitive information. The final design uses smaller inputs, redaction, file limits, batching, intermediate summaries, and warnings. This adds complexity, but it better fits the purpose of a security-focused tool.

The most important remaining risks are documented as accepted issues rather than hidden. Prompt injection is reduced through data boundaries and response validation, but it is not fully solved. Secret detection is useful, but regular expressions can miss unknown or unusual key formats. Dependency checks are present, but the project does not yet include stronger supply-chain controls such as SBOM generation. AI provider rate limits and server errors are handled with retries and fallback paths, but long report generation can still be slow or inconsistent.

A final security point is that the generated report itself must be protected. The report may contain architecture details, risky files, trust boundaries, and likely weak points. Even without raw secrets, that information is valuable. The report should be treated as a sensitive engineering document and should not be committed publicly or shared broadly without review.

The next version of the project should focus on improving evidence quality and reducing residual risk. The highest-priority improvements are an adversarial prompt-injection test suite, a stronger local secret scanner, supply-chain provenance checks, and better handling of AI rate limits. Longer-term work could add SARIF export, IDE integration, and a review mode that helps security teams compare findings across multiple repositories.

Overall, the final release meets the main engineering goal of the course project. It is buildable, documented, tested, and designed around explicit trust boundaries. Its main value is not that it automatically proves a repository is secure. Its value is that it gives developers a structured first pass over a codebase and its GitHub work queue, while keeping the workflow narrow enough for a human to review and control.